

Foundations of Artificial Intelligence - 2026.1

Sequential Decisions under Risk - General Policy Iteration

Felix Mohr

Universidad de La Sabana

Chía, April 7, 2026

Decision Making Situations

Decision Situation Matrix

		Action Effects		
Horizon		Certainty	Risk	Uncertainty
	Atomic	✓	✓	
	Sequential	✓	NOW: 75%	

Class Overview

1. Value Iteration
2. Policy Iteration with Arbitrary Incremental Policy Evaluation
3. General Policy Iteration for known MDPs

Value Iteration

The Bellman Optimality Equation as Update Rule

When we proved that no local optima exist in policy iteration, we used the Bellman optimality equation:

$$v(s) = r(s) + \gamma \max_a \sum_{s'} \mathbb{P}(s'|s, a) v(s')$$

and noticed that, as an operator, it would be a contraction with fix point $v = v^{\pi^*}$.

But if this is a contraction, we could use it as an alternative to PI and simply update state values until we converge to v .

The algorithm that does this and returns the greedy policy is called **Value Iteration (VI)**.

Value Iteration

Optimality Gap Bound

Analogously to iterative PE only replacing the fix point v^π by v , we get that

$$\|\hat{v}_t - v\|_\infty \leq \|\hat{v}_t - \hat{v}_{t-1}\| \cdot \frac{\gamma}{1 - \gamma}$$

So we can stop VI if $\|\hat{v}_t - \hat{v}_{t-1}\| < \varepsilon \frac{1-\gamma}{\gamma}$ and be sure that $\|\hat{v}_t - v\|_\infty < \varepsilon$.

Observe the fundamental difference to incremental PE:

- In incremental PE, we only had an estimation error bound of v^π .
- Here, we have such a bound for the globally optimal v , in particular $v(s_0)$.

Value Iteration

The Algorithm

- 1: Initialize $\hat{v}$ randomly, with values $r(s)$ in any state s
- 2: **repeat**
- 3: $\hat{v}' \leftarrow \hat{v}$
- 4: $\hat{v}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} \mathbb{P}(s'|s, a) \hat{v}'(s')$
- 5: **until** $\|\hat{v} - \hat{v}'\|_\infty < \varepsilon \frac{1-\gamma}{\gamma}$ or solving the linear system indicates optimality
- 6: **return** $\pi : s \mapsto \arg \max_a \sum_{s'} \mathbb{P}(s'|s, a) \hat{v}(s')$

Alternatively, if $\|\hat{v} - \hat{v}'\|_\infty$ is small enough and we have the capacities, we can compute the exact state values once for the current policy and see whether it has converged.

Value Iteration

VI as Variant of PI

The $\hat{v}$ in VI seem to always estimate v without knowing π^* .

While this seems like completely different to PI, there is an alternative view.

PI with iterative PE with a *single* Bellman update (initialized at $\hat{v}^{\pi_t}$) yields

$$\hat{v}_t^{\pi_t}(s) \leftarrow r(s) + \gamma \sum_{s'} \mathbb{P}(s'|s, \pi_t(s)) \hat{v}_{t-1}^{\pi_{t-1}}(s')$$

$$\pi_t(s) = \arg \max_a \sum_{s'} \mathbb{P}(s'|s, a) \hat{v}_{t-1}^{\pi_{t-1}}(s')$$

Plugging in the definition of π_t into the first rule, we obtain the VI update rule:

$$\hat{v}_t^{\pi_t}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} \mathbb{P}(s'|s, a) \hat{v}_{t-1}^{\pi_{t-1}}(s')$$

Value Iteration

Disruption by VI in the Interpretation of the Meaning of $\hat{v}$

This new viewpoint reveals an interesting double interpretation of the $\hat{v}_t$ in each round:

- In the PI view, these are one-step estimates $\hat{v}_t^{\pi_t}$ for the $\hat{v}_{t-1}$ -greedy policy π_t
- In the VI view, these are simply estimates $\hat{v}$ for an optimal policy π^* .

So applying the same update formula to convert the same values $\hat{v}_t$ to some $\hat{v}_{t+1}$ is an update of the state value estimates of two different policies π_t and π^* *simultaneously*.

Any vector can be seen as an estimate of both, but only some have special properties.

Class Overview

1. Value Iteration
2. Policy Iteration with Arbitrary Incremental Policy Evaluation
3. General Policy Iteration for known MDPs

Policy Iteration with Arbitrary Incremental Policy Evaluation

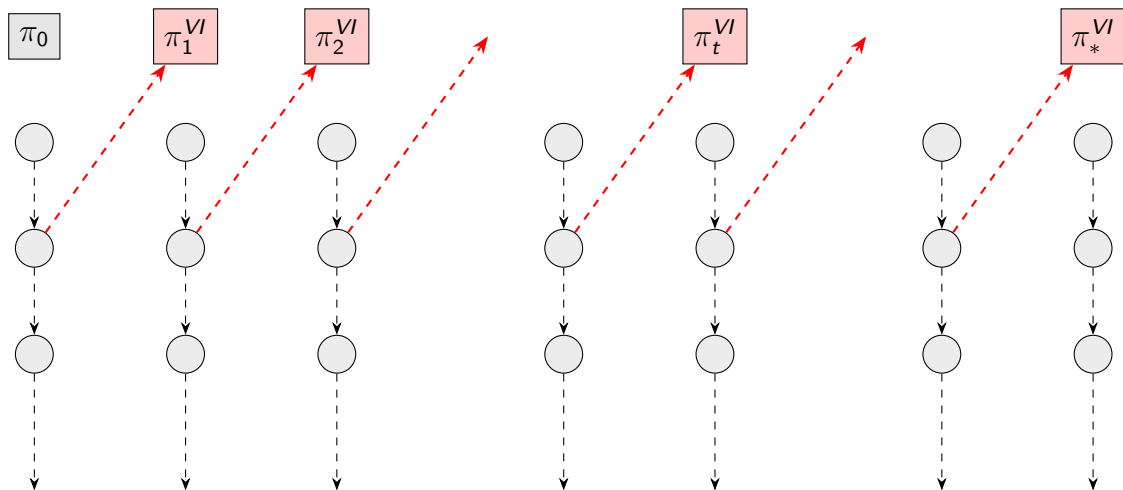
Disruption by VI at Guarantee Level

One of the most revolutionary insights about VI is that the PIT might be obsolete since

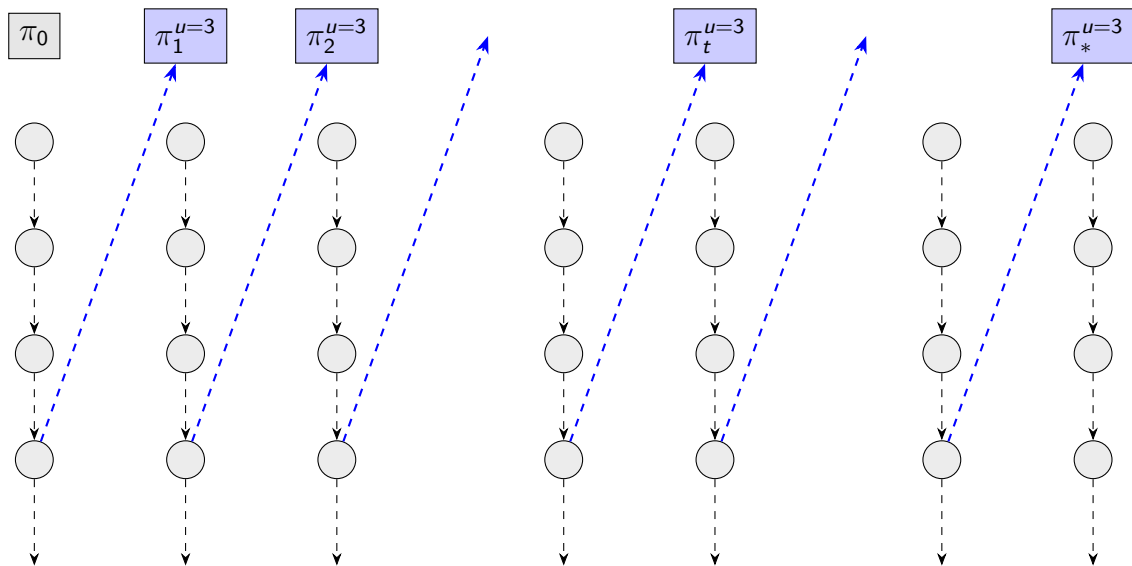
- even though VI is an extreme version of PI with 1-step incremental PE,
- it is guaranteed to converge to state values of an optimal policy.

This suggests that we can do just any *arbitrary* number $u \geq 1$ or even a random number of Bellman updates in PE and still always converge to an optimal policy.

Policy Iteration with Arbitrary Incremental Policy Evaluation



Policy Iteration with Arbitrary Incremental Policy Evaluation



Policy Iteration with Arbitrary Incremental Policy Evaluation

The Combined Operator

Let us now interpret Policy Iteration in the way of state-value updates. Then let

$$\hat{v}_{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} \mathbb{P}(s'|s, a) \tilde{v}_t(s')$$

where $\tilde{v}_t$ is the result of u consecutive Bellman updates initialized with $\hat{v}_t$.

This is called the **combined operator** of length u , because it combines the standard Bellman operator and the Bellman optimality operator.

Policy Iteration with Arbitrary Incremental Policy Evaluation

The Combined Operator

As in VI, the iterates can be interpreted both as partial evaluations of the current greedy policy π_t and as successive approximations to the optimal value function v .

For this reason, we again do not use π_t in the notation for $\hat{v}$.

In contrast to VI, we must make the policy π_t explicit again to perform the u PE steps.

Policy Iteration with Arbitrary Incremental Policy Evaluation

The Combined Operator - A Contraction

It is easy to see that the combined operator is also a contraction with constant γ^{u+1} :

$$\|Ca - Cb\|_{\infty} \leq \gamma^{u+1} \|a - b\|_{\infty}$$

Repeatedly applying the combined operator converges to the true optimal state values.

So the Policy Improvement Theorem for PI with incremental PE is obsolete.

In fact, π_{t+1} will often violate the condition for policy improvement over π_t , and in fact represent a deterioration. The contraction guarantees convergence in the limit.

Policy Iteration with Arbitrary Incremental Policy Evaluation

Modified Policy Iteration

Using the combined operator for PE is often called **Modified Policy Iteration**:

1. start with $\hat{v}(s) = r(s)$ on every final state s and other $\hat{v}(s)$ random or 0.
2. compute $\pi(s) := \arg \max_a \sum_{s'} \mathbb{P}(s'|s, a) \hat{v}(s')$
3. update $\hat{v}$ using u incremental PE-steps initialized with $\hat{v}$ and using π .
4. stop if $\hat{v}$ has changed no more than $\varepsilon \frac{1-\gamma^{u+1}}{\gamma^{u+1}}$ in any state; otherwise go to step 2.

The expected utility of the returned policy π deviates by at most ε from that of π^* .

Note that u can here even changed across iterations without losing any property.

Class Overview

1. Value Iteration
2. Policy Iteration with Arbitrary Incremental Policy Evaluation
3. General Policy Iteration for known MDPs

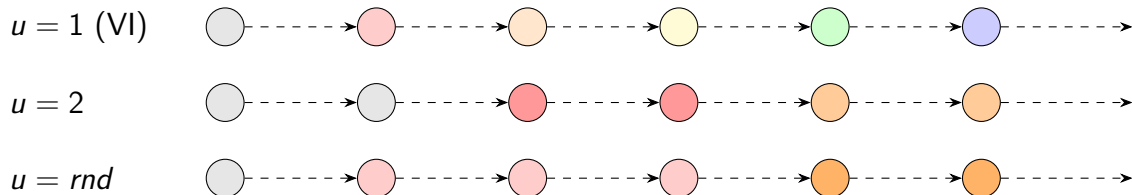
General Policy Iteration

Disruption by VI at Algorithmic Level

VI and MPI motivate a radically different view onto PI in general.

- Before: Repeat PImp using the v^π estimates returned by PE
- Alternative: Repeat Bellman updates and (sometimes) update π with PImp

This figure illustrates the Bellman update sequence (colors $\equiv$ policies):



General Policy Iteration

So incremental PE and greedy Plmp is equivalent to a Bellman update sequence with occasional changes in the policy, and any such sequence converges to the optimal policy.

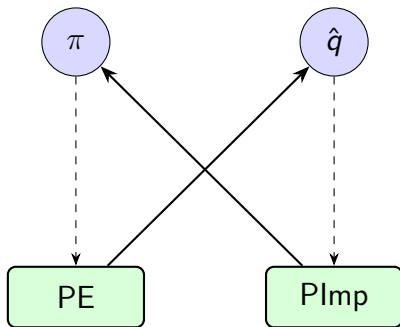
But what if we want to use other techniques for PE or PI?

- PE: We could try to *learn* v^π from data.
- Plmp: We could use *lookaheads* (rollouts) instead of only locally looking at $\hat{q}$.

Does the Bellman-update sequence idea carry over to a more general version of PI?

General Policy Iteration

A general architecture could have two components operating on two shared resources:



PE of course also has read access to $\hat{q}$.

General Policy Iteration

Sparse State-Action Values in the Communication Layer

The architecture assumes that PE updates state-action values $\hat{q}$.

This is more general than only $\hat{v}$, because these are contained.

In addition, we allow that $\hat{q}$ is sparse (not need to be defined for all state-action pairs).

Of course, PImp must treat these missing values, e.g., picking a random action.

Advantage: We can even run PE and PImp in *parallel* so that PImp can improve the policy even before PE has assessed the quality of all actions in all states.

General Policy Iteration

Asynchronous Updates - Asynchronicity of PE and PI

GENERALPOLICYITERATIONFORMDPs(*mdp*, γ , *PolicyEvaluationFactory*, *PolicyImprovementFactory*)

- 1: $\hat{q} \leftarrow \emptyset$ // shared sparse object
- 2: $PImp \leftarrow PolicyImprovementFactory(mdp, \gamma, \hat{q})$
- 3: $\pi \leftarrow PImp.\pi$ // make policy a shared object
- 4: $PE \leftarrow PolicyEvaluationFactory(mdp, \gamma, \hat{q}, \pi)$
- 5: **repeat**
- 6: $PE.update_q()$
- 7: $PImp.update_pi()$
- 8: **until** no more iterations
- 9: **return** π

This is a sequential implementation. Alternatively, one can run 2 processes with $PE.update_q()$ and $PImp.update_pi()$ respectively.

General Policy Iteration

Why Policy Iteration Still Converges to the Optimal Policy

General Policy Iteration cannot converge to a sub-optimal policy as long as

- PE eventually converges $\hat{q}$ to q^π if Plmp does not change the policy π
- Plmp eventually greedily updates π if an advantage has been detected

Proof Sketch:

- Suppose it converges towards sub-optimal policy π
- Since π is sub-optimal, it does not satisfy the Bellman equation, so there is some s, a so that $q^\pi(s, a) > q^\pi(s, \pi(s))$.
- Since we converge towards π , we realize an infinite number of PE steps for π , so $\hat{q}$ converges towards q^π , detecting an advantage $\hat{q}(s, a) - \hat{q}(s, \pi(s))$
- Plmp eventually updates π , which contradicts convergence towards π .

Summary

Summarizing the lecture in one phrase, we saw that

- an extreme case of PI is with 1-step incremental PE, called Value Iteration,
- which converges to π^* , making PIT obsolete, and allowing to configure an arbitrary number u of updates,
- motivating an even more general version of PI in which we can arbitrarily update q-values and the policy as long as we converge to true q-values and eventually do greedy updates on π .